Task: To-Do List Web App

Your task is to create a simple To-Do List web app where users can add, view, complete, and delete tasks. The project combines frontend development (HTML, CSS, JavaScript/React) with a backend server (Flask) and connects them through API endpoints.

Main Requirements

1. Frontend:

Basic structure (HTML):

- Add a heading (for example: "My To-Do List").
- Add an input box and a button to add new tasks.
- Add an empty list using either <ul> (unordered list) or a .
- Add a button to switch between Light Mode and Dark Mode.

A (division) is a generic container element in HTML. It does not have any meaning by itself but is used to group other elements together to control the structure better. You can style it or target it with JavaScript as any other element.

Styling (CSS):

- Make the layout look clean and readable.
- Define two themes: Light Mode: white background, black text.
  Dark Mode: black background, white text.
- Use CSS classes such as **.light-theme** and **.dark-theme** so that you can easily switch between them with JavaScript.

Basic Functionality (JavaScript):

- When you type a task and click "Add", it should show up in the list.
- Add a button that switches the page between Light and Dark themes.

Hint: you can use the following JavaScript methods:

document.getElementById to find elements.
innerHTML or **appendChild** to add tasks
classList.toggle to switch themes.

2. Backend:

- **Framework**: Use **Flask** to implement the backend server.
- Create the necessary API endpoints to handle all the functionalities required in the frontend part

- **CORS Support**: Enable CORS (Cross-Origin Resource Sharing) to allow communication between the React frontend and Flask backend.
- **Error Handling:** Return appropriate error responses for any issues, like invalid inputs or server errors.

Resources:

How to Make an API Request in React
In React, you can use libraries like Axios or the Fetch API to make requests to your backend.
Axios: Axios in React: A Guide for Beginners - GeeksforGeeks
Fetch: Using the Fetch API - Web APIs | MDN (mozilla.org)
Flask Documentation:
Main Documentation: https://palletsprojects.com/projects/flask/
Miguel Grinberg's Flask Mega-Tutorial: https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-i-hello-world

Bonus (Optional):

- Make tasks clickable so that clicking a task marks it as completed (for example, by applying a line-through style).
- Add a small "Delete" button next to each task to remove it.
- Save tasks in localStorage so that they remain even after refreshing the page.

Submission Details:

1. Create a new repository called "yourname_AlexEagles_Phase2"
2. Make a new branch called " To-Do-List_WebApp" and upload your files on it
3. Submit a .txt file with the link to your repo in the following form for the backend task: https://docs.google.com/forms/d/e/1FAIpQLSe1c_qawhTi_HoLrkVk37cUyVr3TrHWJT7JL_1Zytfaa_mmSw/viewform
4. Deadline is 20/9 11:59 pm